

Redis Cluster Scale-In Logic and Limitations

Purpose: Scale-in removes the dynamic scale-out pair from the Redis Cluster and returns the topology to the original six-node base cluster.

Supported command: `./redis-tool scale --remove-node 10.10.0.17` or `./redis-tool scale --remove-node 10.10.0.18`

Expected final topology: 3 masters, 3 replicas, 16384 hash slots covered, and all verification checks passing.

1. What Scale-In Is Designed To Remove

The project has a fixed six-node base Redis Cluster and one supported scale-out pair.

Node group	IPs	Scale-in behavior
Base cluster	10.10.0.11 through 10.10.0.16	Protected. These nodes cannot be removed by scale-in.
Scale-out pair	10.10.0.17 and 10.10.0.18	Supported. The tool removes this pair and verifies the cluster.

2. High-Level Flow

- 1 The main redis-tool command loads shared modules, checks Docker or Podman, checks Ansible, and opens an operation log.
- 2 The scale command dispatches `--remove-node` to the scale-in handler.
- 3 The handler validates the argument and rejects base cluster IPs.
- 4 The handler confirms that the existing base cluster is healthy before modifying membership.
- 5 The tool reads redis-cli cluster nodes from redis-node-1 and identifies the scale-out master and replica pair.
- 6 If a master is being removed, the Ansible scale-in playbook migrates its hash slots to remaining healthy masters.
- 7 The wrapper verifies that the removed master owns no hash slots.
- 8 The replica is removed from cluster membership first, then the master is removed.
- 9 The related scale-out containers are stopped and removed.
- 10 The full verification playbook runs to confirm cluster health, topology, replication, Redis versions, and data integrity.

3. Detailed Scale-In Logic

Command validation

The command must be exactly:

```
./redis-tool scale --remove-node <node-ip>
```

The IP must match the internal Redis network format 10.10.0.x.

Base-node protection

The script refuses to remove 10.10.0.11 through 10.10.0.16. This is intentional because the base topology is the minimum supported cluster layout for this project.

```
[ERROR] Cannot remove base cluster nodes (10.10.0.11-16). Only scale-out nodes (10.10.0.17-18) can be removed
```

Pair identification

The script uses the Redis Cluster node table to map the requested IP to a node ID and role.

- If the target is a master, it finds the healthy replica whose fourth field points to that master ID.
- If the target is a replica, it finds the master ID from the replica line and then finds the master IP.
- If the pair is only partially present, it handles cleanup of the remaining scale-out node as long as that node owns no hash slots.
- If both scale-out nodes are already absent, the operation is treated as already complete and verification still runs.

Slot migration

For a normal scale-in, the target master must give up all hash slots before deletion. The playbook runs a Redis Cluster rebalance with the target master weight set to zero:

```
redis-cli --cluster rebalance 10.10.0.11:6379 \  
--cluster-weight "<target_master_id>=0" \  
--cluster-use-empty-masters \  
--cluster-yes
```

Membership removal

- 1 Remove the replica with `redis-cli --cluster del-node`.
- 2 Remove the master with `redis-cli --cluster del-node`.
- 3 Stop and remove the matching scale-out containers.

Verification

The operation runs `ansible/playbooks/verify_full.yml`. A successful final state looks like:

```
[PASS] Cluster state is ok  
[PASS] All 16384 hash slots are covered  
[PASS] All 6 cluster nodes are present and reachable  
[PASS] All 6 nodes are running the same Redis version: 7.2.6  
[PASS] Topology has 3 masters and 3 replicas; every master has a replica  
[PASS] All 3 replicas have master_link_status:up  
[PASS] Data integrity verified: 1000/1000 keys matched  
FULL VERIFICATION RESULT: PASS
```

4. What Cannot Be Scaled In

- **Base cluster nodes cannot be removed:** 10.10.0.11 through 10.10.0.16 are protected.
- **Arbitrary extra nodes are not supported:** the current tool only knows the dynamic pair 10.10.0.17 and 10.10.0.18.
- **A scale-out master with no healthy replica is not removed in the normal path:** scale-in is meant to remove a safe master-replica pair.
- **A remaining scale-out node that still owns slots is not removed by the partial-cleanup path:** the tool stops rather than risking slot loss.
- **An unhealthy base cluster cannot be modified:** scale-in requires the base cluster to be reachable and healthy first.

5. Current Limitations

- The topology is intentionally fixed: 3 base masters and 3 base replicas.
- Scale-out supports exactly two added nodes: one new master and one new replica.

- Scale-in supports only the dynamic scale-out pair, not general cluster shrink operations.
- The IP mapping is static: redis-node-7 uses 10.10.0.17, and redis-node-8 uses 10.10.0.18.
- The tool is built for this assignment topology, not for arbitrary Redis Cluster production resizing.
- The workflow depends on Docker or Podman, Ansible, SSH reachability, and Redis CLI availability inside the containers.

6. Issues Faced and Fixes

Issue 1: Replica lookup failed

During an early run, the tool removed 10.10.0.17 but failed to remove the paired node cleanly. The root cause was brittle parsing of redis-cli cluster nodes. The old lookup expected the master ID to appear at the end of the replica line, but Redis lines contain additional fields after the master ID.

Fix: parse the cluster table by fields and use field 4 as the replica master ID.

Issue 2: Remaining scale-out node after partial removal

After the master was removed, 10.10.0.18 remained in the cluster and was reattached as a replica of a base master. The original recovery logic only handled a replica whose master was missing, so it did not clean up this case.

Fix: if one scale-out peer is already absent, the tool checks the remaining peer. If the remaining peer owns no hash slots, it safely removes it and stops its container.

Issue 3: Retry after successful cleanup returned failure

After both scale-out nodes were gone, rerunning `./redis-tool scale --remove-node 10.10.0.17` still returned an error because the requested node was not found.

Fix: scale-in is now idempotent for 10.10.0.17 and 10.10.0.18. If both scale-out nodes are already absent, the tool reports the operation as already complete, runs full verification, and exits successfully.

7. Final Behavior After Fixes

Command	Expected result
<code>./redis-tool scale --remove-node 10.10.0.17</code> when pair exists	Migrate slots, remove replica and master, stop containers, verify cluster.
<code>./redis-tool scale --remove-node 10.10.0.17</code> when only 10.10.0.18 remains	Remove remaining scale-out node if it owns no slots, stop container, verify cluster.
<code>./redis-tool scale --remove-node 10.10.0.17</code> when both scale-out nodes are gone	Report already complete, run verification, exit successfully.
<code>./redis-tool scale --remove-node 10.10.0.12</code>	Fail safely because 10.10.0.12 is a protected base node.

8. Recommended Operational Rule

Use scale-in only to undo the supported scale-out operation. In this project, that means removing the pair 10.10.0.17 and 10.10.0.18. Do not use scale-in to shrink the base six-node cluster.